

Activité : auditer un script DESeq2 généré par IA

Activité formative, non notée

Contexte

Une doctorante du laboratoire veut savoir quels gènes la dexaméthasone induit dans les cellules musculaires lisses des voies respiratoires. Elle dispose des données `airway` :

- `airway_counts.tsv` : les comptes RNA-seq, une ligne par gène, une colonne par échantillon ;
- `airway_sample_info.tsv` : une ligne par échantillon, dont `Run` (identifiant), `cell` (la lignée, c'est-à-dire le donneur) et `dex` (`trt` traité, `untrt` non traité).

Elle a demandé à un assistant d'IA :

« J'ai 4 lignées de cellules musculaires lisses, chacune traitée ou non à la dexaméthasone (airway). Écris un script DESeq2 qui compte les gènes différentiellement exprimés ($FDR < 5\%$) et liste les 10 gènes les plus induits par le traitement. »

Voici le script obtenu. Il a été écrit pour cet exercice, dans le style d'un assistant d'IA. **Il s'exécute sans aucune erreur.**

```
library(tidyverse)
library(DESeq2)
# Script vérifié : s'exécute sans erreur.
comptes <- read_tsv("/home/public/stat/airway_counts.tsv") |>
  column_to_rownames("gene") |> as.matrix()
infos <- read_tsv("/home/public/stat/airway_sample_info.tsv") |>
  column_to_rownames("Run")

dds <- DESeqDataSetFromMatrix(countData = comptes, colData = infos,
                              design = ~ dex)
dds <- dds[rowSums(counts(dds)) >= 10, ]
dds <- DESeq(dds)
res <- results(dds)

# 1. Nombre de gènes différentiellement exprimés (FDR < 5 %)
res_df <- as.data.frame(res)
nrow(res_df[res_df$padj < 0.05, ])
```

```
# 2. Les 10 gènes les plus induits par la dexaméthasone
res_df |>
  filter(padj < 0.05) |>
  arrange(desc(log2FoldChange)) |>
  head(10)
```

Le script contient **au moins 3 erreurs**. Aucune ne produit d'erreur : les résultats ont l'air normaux, mais ils sont faux. Attention, certaines lignes sont **correctes** : ne corrigez que ce qui doit l'être.

Partie A : lire sans exécuter

1. Pour chaque bloc, écrivez en une phrase ce que le code est **censé** faire, puis ce qu'il fait **réellement**.
2. Relisez la **demande** : que sait-on du plan d'expérience ? Le **design** en tient-il compte ?
3. Dans **infos**, la colonne **dex** contient du texte : **"trt"** et **"untrt"**. Quel sera le **niveau de référence** de la comparaison ? Que représentera donc un log2FoldChange positif ?
4. Que fait le crochet `res_df[res_df$padj < 0.05, ]` d'une ligne où **padj** vaut **NA** (module 5) ? Et `filter()` ?
5. Le commentaire affirme « Script vérifié ». Qu'en pensez-vous ?

Partie B : vérifier dans RStudio

1. Exécutez le script et notez les résultats. Lisez **tous** les messages affichés, y compris les *warnings*.
2. Avant de corriger, **inspectez** :

```
res                                # la première ligne : quel contraste ?
sum(res$padj < 0.05, na.rm = TRUE) # le même compte, calculé autrement
sum(is.na(res$padj))               # combien de gènes non testés ?
dplyr::count(infos, cell, dex)     # le plan d'expérience
```

3. Quels soupçons de la partie A se confirment ?

Partie C : corriger

1. Corrigez le script pour qu'il réponde vraiment à la demande.
2. Pour chaque correction, ajoutez un commentaire (**#**) qui explique l'erreur.
3. Comparez vos résultats avec ceux du script d'origine : de combien diffèrent-ils ?

Pour finir : discussion

- Quelles vérifications auriez-vous dû faire **avant** de faire confiance à ce script ?

- Les erreurs de cette semaine viennent-elles du code, des données ou du plan d'expérience ? Qui aurait dû les repérer ?
- Qu'auriez-vous pu écrire dans la demande à l'assistant pour les éviter ?

